BM25 vs. Neural Retrieval on Technical Documents

Vishrut Bhushan	h20250143@pilani.bits-pilani.ac.in
Bhagavath M S	h20250167@pilani.bits-pilani.ac.in
Ajay B	h20250142@pilani.bits-pilani.ac.in
Jalluri Mahesh Kumar	h20250061@pilani.bits-pilani.ac.in

1. Primary Objective

In the field of Information Retrieval, the need for better retrieval algorithms is never ending. No one can claim to have found the “Golden” retrieval algorithm because such a thing cannot technically exist. Each algorithm or system of algorithms comes with its own set of pros and cons. This is exaggerated by the need to continuously keep adapting to the everchanging world of information where data is getting generated in Terabytes per second. What is state of the art today might become irrelevant tomorrow.

In our project, we have chosen to pick a specific topic of comparing the established and robust classic algorithms like BM25 which try to preserve the lexical analysis and put them head to head in a face off against newer AI centric neural retrieval algorithms which focus more on the semantic aspect of the queries. We will be putting them to test and getting quantified metrics to provide a definite answer at least to the questions we are looking for.

The type of data which we want to handle impacts the architectural and implementation choices by a lot. So we have tried to settle for a data corpus consisting of web pages (HTML dumps), from several technical websites in the world of Software and AI. This domain is far more interesting than picking simplistic library style datasets and comes with its own nuances .

This will help us get an accurate representation of how the search engines like Google,Bing etc work in real time on a similar but gigantic scale.

Based on the outcomes, we will also try coming up with novel enhancements to the existing algorithms which aim to improve the performance of said algorithms.

2. Corpus

Primary source: Common Crawl. (Common Crawl – Get Started | URL Index) : Stores raw web pages in WARC and provides a URL Index for bulk filtering of crawl records.

Filters: Programming Languages, Coding Frameworks, Cloud Computing and Infra, Distributed Computing and Data, Databases, DevOps, Observability, Networking, ML Libraries etc.
Document form: Each item will remain an HTML document with URL, title, heading hierarchy,

paragraphs, lists, tables, code blocks, links/anchor text and metadata. The raw HTML is retained.

Corpus Size: 79,275 Files : 18.6 GB (20,073,509,815 bytes). We might have to trim down a little based on the compute available.

3. Background Research

3.1 Preprocessing and Query Representation

Problem: Boilerplate, navigation, markup, variable terminology, filler & stop words.
Existing solutions: Web-text extraction methods remove boilerplate content (Kohlschütter et al., 2010). Classical IR uses tokenization, normalization, stemming such as the Porter algorithm, and stop-word handling. **k-gram indexes** support spelling correction and wildcard matching, while query expansion can use lexical-semantic relations such as synonyms (Voorhees, 1994). For technical corpora, identifiers, code tokens and version strings require less aggressive normalization.

3.2 Lexical Representation and Ranking

Problem: Exact term matching, rare terms, varying length
Existing solutions: IDF provides greater weight to rarer terms (Spärck Jones, 1972), while TF-IDF combines term frequency and collection frequency (Salton & Buckley, 1988). **BM25**'s probabilistic relevance, term-frequency saturation and document-length normalization, building on Okapi at TREC-3 and formalized by Robertson & Zaragoza (2009).

3.3 Zones and Structured Indexing

Problem: Non uniform distribution of weightage across sections/tags.
Existing solutions: Zone/field indexes and weighted-zone scoring assign different importance to fields. BM25F extends BM25 to multiple fields (Robertson & Zaragoza, 2009). Structured and hierarchical indexes have also been studied for exploiting document hierarchy rather than treating each document as flat text (Cui et al., 2003).

3.4 Index Efficiency

Problem: Exhaustive postings traversal are expensive.
Existing solutions: Classical retrieval uses skip structures, tiered/champion indexes, multi-level indexes and document clustering to reduce the candidate space.

3.5 Neural / Dense Retrieval

Problem: Lexical methods can miss semantically related passages
Existing solutions: DPR introduced dual-encoder dense retrieval, mapping queries and passages into a shared vector space. ANCE improved dense training through hard-negative mining, while E5 and more recent Qwen3-Embedding develop increasingly general-purpose embedding models.

3.6 Structure-Aware and Multi-Vector Retrieval & Chunking

Problem: Flat passage embeddings can lose document-level context, while a single vector may compress too much information.
Existing solutions: Dense Hierarchical Retrieval combines document- and passage-level retrieval using hierarchical structure. ColBERTv2 retains multiple contextual representations through late interaction, providing finer-grained matching.

3.7 Weighting, Ranking and Caching

Problem: Non optimal fixed scoring parameters across document zones, query types and retrieval signals, while repeated queries introduce unnecessary computation.
Existing solutions: Classical retrieval supports **parametric scoring and weighted-zone models**, where field weights and ranking parameters can be tuned. Multi-stage ranking can also combine lexical, semantic and structural signals. Search-engine research demonstrates that **query-result caching** can exploit query locality to reduce repeated computation (Markatos, 2001); later systems also use multi-level caching of results, postings and precomputed scores (Ozcan et al., 2012).

4. Proposed Approach

4.1 Data Layer

The original Common Crawl HTML documents remain unchanged. A derived representation is created containing URL, title, headings, body, code, links, metadata and document hierarchy.

4.2 Common Processing Layer

The corpus passes through HTML parsing, boilerplate removal, tokenization and normalization. Technical identifiers are preserved. The processor produces both document-level and section/passage-level representations so that flat and structure-aware retrieval can be evaluated from the same corpus.

4.3 Classical Retrieval Layer

A multi-field inverted index is constructed for title, headings, body, code and other useful zones, with positional information where required. The retrieval stack contains: **TF-IDF ~> BM25 ~> weighted-zone/BM25F variants** .A hierarchical or multi-level index can additionally support document, section, passage retrieval. Parametric weights and scoring parameters will be configurable and can be tuned independently.

4.4 Neural Retrieval Layer

The same structured passages are encoded using a pretrained dense embedding model such as E5 or Qwen3-Embedding. Embeddings are stored in an ANN vector index for top-k retrieval. A

structure-aware variant retains title and heading context, with hierarchical retrieval or multi-vector retrieval considered as extensions.

4.5 Query Processing Layer

A common query processor handles normalization and optional modules such as spelling correction, synonym expansion, phrase matching and proximity matching. The resulting query is submitted to the classical and/or neural retrieval pipelines.

4.6 Ranking and Weighting Layer

The architecture supports a second-stage ranking layer combining signals such as: **BM25 score + dense similarity + zone weights + structural relevance.** Weights and scoring parameters can initially be configured parametrically and, where useful, learned from relevance judgments. This permits comparison between manually weighted, parameter-tuned and learned ranking strategies.

4.7 Caching and Efficiency Layer

A caching layer stores **frequent query results, processed queries and query embeddings** where applicable. Index-level optimizations such as multi-level indexes, clustering and Bloom-filter filtering can be evaluated independently. Latency, memory, index size and indexing time are recorded alongside retrieval effectiveness.

5. Query Design and Evaluation Metrics

To provide a controlled comparison between the **lexical** and **dense neural retrieval approaches**, a common set of queries and evaluation criteria will be used for both systems. The evaluation will focus not only on overall retrieval effectiveness but also on how the two approaches behave under different types of technical information needs.

5.1 Query Design

A set of 20 complex technical queries will be constructed to evaluate the retrieval behavior of **BM25** and **dense neural retrieval**. The queries will cover different retrieval challenges, including **exact technical terminology**, **paraphrasing**, **negation or exclusion requirements**, **homonyms** and **ambiguous terminology**, **temporal/version constraints**, and **combinations of multiple constraints**. The query set will be designed to represent realistic information needs in technical documentation rather than relying only on simple keyword-based queries.

The queries will be evaluated using the same corpus and relevance judgments for both retrieval approaches. No arbitrary weighting will be assigned to different query categories in the baseline evaluation. Instead, results will be analyzed both overall and by query type to identify cases where lexical or semantic retrieval performs differently. Specific handling of challenging cases such as negation may be investigated later as a retrieval enhancement rather than being incorporated into the baseline comparison.

5.2 Evaluation metrics

The primary retrieval effectiveness metrics will be **Normalized Discounted Cumulative Gain at 5 (NDCG@5)** and **Mean Average Precision (MAP)**, as required for the comparative evaluation. NDCG@5 measures the quality and ranking of the top five retrieved results, while MAP measures the overall effectiveness of retrieving relevant documents across the ranked results.

Recall@20 measures how many relevant documents are retrieved in the top 20 results, **Precision@20** measures the proportion of the top 20 results that are relevant, and **MRR** measures the rank of the first relevant result. Query latency and index size may also be recorded where feasible.

The evaluation approach will be over **BEIR**, a standard benchmark for comparing different retrieval methods. However, this project will use its own technical-web corpus, 20 custom queries, and relevance judgments.

6. Experiments & Innovations

These are some of the proposed experiments to understand further improvement in the existing techniques from the given algorithms and develop innovations based on the results.

Area	Proposed Extension	Expected Effect
Classic IR	Query-adaptive HTML-zone BM25	Improve ranking by giving more weight to the zones most informative for the query type.
Ablation Study	Compare flattened document retrieval v HTML based retrieval	Quantify how much of the improvement over retrieval is achieved from HTML structure rather than that of the algorithm or model itself.
Hybrid IR	BM25+dense score fusion: Combine lexical BM25 and semantic similarity score as additional retrieval strategy	Experimental approach to see how well semantic and lexical matching complement each under complex queries
Strategy	Query type aware IR:	Stress testing the respective algorithms under different queries

Area	Proposed Extension	Expected Effect
	Evaluate retrieval separately for exact terms, negation, ambiguity or multi constraint queries	
Neural IR	HTML-aware hierarchical dense retrieval	Preserve document context while retrieving fine-grained relevant sections; target is better NDCG/MAP.
Neural IR (optional)	Zone-aware score fusion	Combine title/heading/body/section similarities rather than relying on one flat representation.

7. Stretch Objectives

- Compare retrieval using Plain text vs HTML structure to see how much document structure improves results.
- Compare the performance of BM25 and dense retrieval across different query types.
- Use Block level bloom filter indexing to skip index blocks that cant contain query terms and evaluate the improvement in latency while retrieval preserving retrieval quality
- Evaluate how retrieval latency , memory usage and index size change as the size of the corpus increases.

8. Milestones

- **Milestone 1 :** Background Research and Corpus finalization
- **Milestone 2 :** Initial Setup and Base paper Implementation on dummy data
- **Milestone 3 :** Dataset Specific changes & pipeline implementation
- **Milestone 4 :** Final comparative evaluation